
Foundations of Deep Learning

Matrix Calculus, Optimization, and Implementation

Anahit Israelyan

May 10, 2026

Abstract

This report walks through the mathematical, algorithmic, and computational ideas that make modern deep learning work. We start with the geometry of affine maps and the calculus of vector-valued functions, then build up to a complete picture of how a neural network learns: how its layers compose into a single differentiable function, how the chain rule flows gradients backward through that function, and how an optimizer turns those gradients into better parameters. Along the way, we cover activation functions and the surprisingly subtle ways they can go wrong, the standard regression and classification losses (and the beautiful gradient that drops out when softmax meets cross-entropy), and a fully worked manual backpropagation example with consistent shape conventions throughout. The second half of the report moves past vanilla stochastic gradient descent into momentum, RMSProp, and Adam; covers initialization (Xavier, He) and the modern regularization toolbox (weight decay, dropout, batch normalization, early stopping); and discusses what we now understand about generalization, including the double-descent phenomenon. The final section ties everything back to actual code via PyTorch and its autograd engine. Every section is paired with a figure that makes the mathematics visible.

Contents

1	Introduction: What Deep Learning Actually Is	4
1.1	A short historical arc	4
1.2	Universal approximation, and why depth still matters	4
1.3	Roadmap	5
2	Mathematical Foundations: Matrix Calculus	6
2.1	The affine transformation	6
2.2	Why non-linearities are mandatory	7
2.3	Jacobians and the vector chain rule	8
2.4	Layout conventions and a worked Jacobian	9
2.5	Batched gradients	10
3	Activation Functions and the Gradient-Flow Problem	11
3.1	The standard menu	11
3.2	The vanishing gradient problem, quantified	11
3.3	ReLU and its pathologies	12
3.4	Mean-centered vs. zero-bounded activations	13
3.5	What about output activations?	13
4	Loss Functions: Quantifying Disagreement	14
4.1	Mean squared error for regression	14
4.2	Cross-entropy for classification	14
4.3	Softmax: turning logits into probabilities	15
4.4	The softmax-cross-entropy gradient	15
4.5	Other losses worth knowing	16
5	Optimization: From Gradient Descent to Adam	17
5.1	Gradient descent on a non-convex landscape	17
5.2	Stochastic gradient descent	17
5.3	Momentum	18
5.4	Adaptive learning rates: RMSProp and Adam	19
5.5	Learning-rate schedules	20
5.6	Pseudocode summary	21
6	The Training Protocol: Forward and Backward Pass	22
6.1	The two passes through the computation graph	22
6.2	Worked example: backpropagation by hand	23
6.3	Automatic differentiation, in one paragraph	24
7	Initialization: Where Training Begins	25
7.1	Why all-zeros doesn't work	25
7.2	Variance preservation across layers	25
7.3	Xavier (Glorot) initialization	25
7.4	He initialization	25
8	Regularization: Fighting Overfitting	27
8.1	L2 weight decay	27
8.2	Dropout	27
8.3	Batch normalization	28
8.4	Early stopping	29
8.5	Data augmentation	29

9	Generalization: The Modern Picture	30
9.1	The classical bias-variance picture	30
9.2	Double descent	30
9.3	Why over-parameterization can help	30
9.4	Practical takeaways	31
10	Implementation: PyTorch and Differentiable Programming	32
10.1	The autograd engine	32
10.2	Modules, optimizers, and the canonical training loop	32
10.3	Memory: training vs. inference	33
10.4	From SGD to LLMs: the same recipe	34
11	Closing Thoughts	35

1 Introduction: What Deep Learning Actually Is

Deep learning is often described as “brain-inspired computation,” but this metaphor causes more confusion than it clears up. What deep learning really is, at its core, is the systematic application of two ideas:

1. **Hierarchical representation learning.** Instead of hand-crafting features — edge detectors for vision, n-gram counts for text, custom kernels for whatever domain — we let a deep parameterized function figure out its own intermediate features directly from raw data. Each layer of the network builds slightly more abstract features on top of the previous one, all of them learned rather than designed.
2. **Differentiable programming.** We write programs that are end-to-end differentiable with respect to their parameters, then we let gradient-based optimization tune those parameters. Where traditional code uses sharp logical control flow (if/else branches, hard loops, discrete masks), differentiable programs smooth everything into continuous tensor operations.

These two ideas reinforce each other. Hierarchical representations are only useful if we can actually train them, and the chain rule applied to a stack of differentiable layers — which is what backpropagation amounts to — is what makes training possible.

1.1 A short historical arc

Rosenblatt’s perceptron [1] could only carve linear decision boundaries through input space, and Minsky and Papert [2] famously pointed out that this excluded operations as basic as XOR. The field stagnated for almost two decades until backpropagation was popularized in the 1980s [3], finally giving us a practical way to train deeper networks. But even then, things didn’t take off immediately. It took three more developments — large labelled datasets like ImageNet, parallel hardware in the form of GPUs, and better choices of initialization, normalization, and activation functions — before deep architectures like AlexNet [5] suddenly beat handcrafted pipelines by huge margins in the early 2010s. That same recipe, scaled up by several orders of magnitude, is what gives us today’s foundation models.

1.2 Universal approximation, and why depth still matters

There is a formal result that explains, in principle, why neural networks are so flexible: the Universal Approximation Theorem [4].

Theorem 1 (Universal approximation, informal). *Let g be any non-constant, bounded, continuous activation function. For any continuous function $f : K \rightarrow \mathbb{R}$ on a compact set $K \subset \mathbb{R}^n$, and any $\varepsilon > 0$, there exists a single-hidden-layer network $\hat{f}$ such that $\sup_{x \in K} |f(x) - \hat{f}(x)| < \varepsilon$.*

The catch is that this is an existence theorem, not a construction. It tells us that *some* shallow wide network can approximate f as closely as we want, but it says nothing about how wide that network has to be, how to find its weights, or whether the result will generalize to new data. In practice, deep networks turn out to express most functions we care about with exponentially fewer parameters than shallow ones. That’s why depth, not just width, is the engineering default.

Intuition

The universal approximation theorem says a single-layer network *could* represent any reasonable function. Depth says the network can represent it *efficiently*, with a parameter count that doesn’t blow up, and with gradients that learn structure compositionally — low-level features in early layers, high-level concepts in late ones.

1.3 Roadmap

Section 2 sets up the matrix-calculus toolkit: affine maps, Jacobians, and the multivariable chain rule. Section 3 examines non-linearities and quantifies the vanishing-gradient problem they introduce. Section 4 derives the standard regression and classification losses. Section 5 covers SGD, momentum, and adaptive methods like Adam. Section 6 pulls everything together with a manual backpropagation walkthrough. Section 7 addresses initialization, Section 8 covers regularization, and Section 9 discusses the modern view of generalization. Section 10 closes with a look at how all of this is implemented in PyTorch.

2 Mathematical Foundations: Matrix Calculus

A deep neural network is, at its heart, a parameterized directed acyclic graph (DAG) of tensor operations. Formally, an L -layer feed-forward network is a composition

$$f_{\theta}(x) = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)}(x), \quad (1)$$

where each $f^{(l)}$ is an affine map followed by a pointwise non-linearity, and $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^L$ collects every trainable parameter. Figure 1 shows a small concrete example.

A 4-layer feed-forward network ($L = 3$ trainable layers)

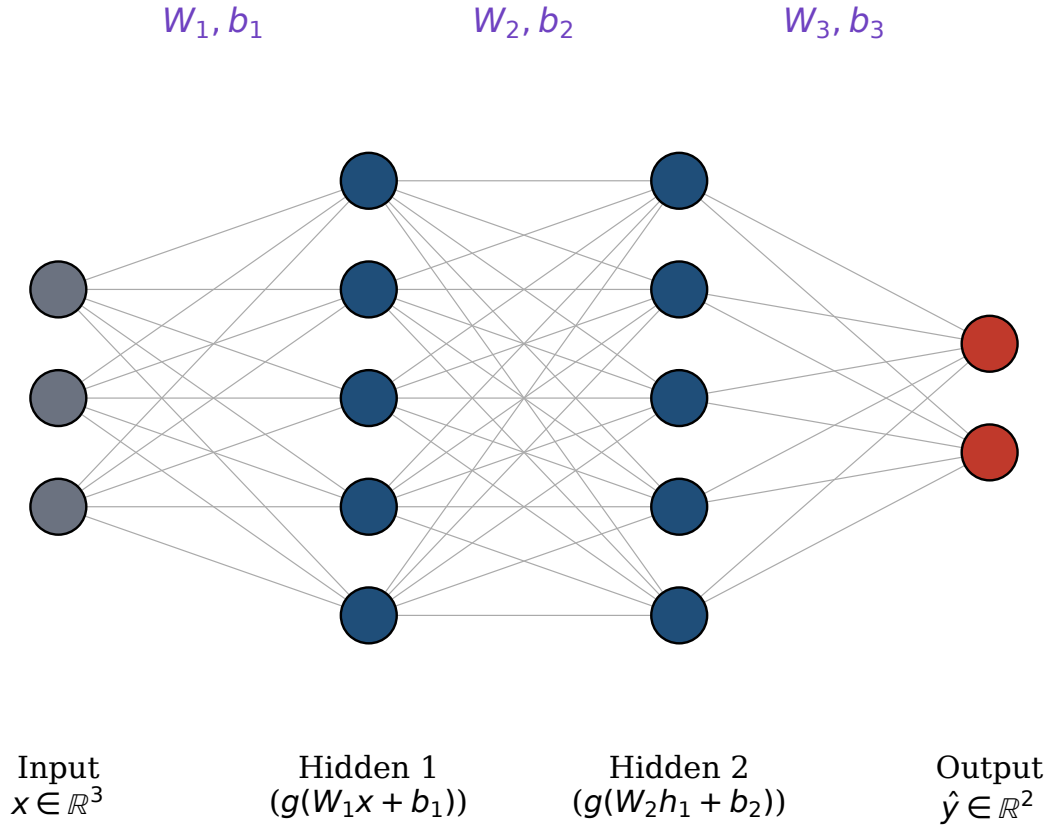


Figure 1: A small feed-forward network. Each node in a hidden layer computes a scalar from a learned linear combination of the previous layer’s activations, then passes the result through a non-linearity. Stacking these gives the function in Equation 1.

To train a network like this we need to compute, for every parameter $W^{(l)}$ and $b^{(l)}$, the partial derivative of a scalar loss $\mathcal{L}$ with respect to that parameter. With millions or billions of parameters to keep track of, we obviously can’t differentiate by hand. We need a systematic, mechanical procedure that produces gradients for every parameter at once. That procedure is backpropagation, and the language it speaks is matrix calculus.

2.1 The affine transformation

The fundamental building block of any feed-forward network is the affine map. Given a vector input $x \in \mathbb{R}^n$, the pre-activation $z \in \mathbb{R}^m$ is

$$z = Wx + b, \quad W \in \mathbb{R}^{m \times n}, \quad b \in \mathbb{R}^m. \quad (2)$$

We say “affine” rather than “linear” because of the bias b : a strictly linear map is forced to send the origin to the origin, which would be a serious limitation. The weight matrix W acts geometrically by rotating, scaling, and shearing the input space, projecting it into an m -dimensional subspace. The bias b then slides the whole result by a fixed offset.

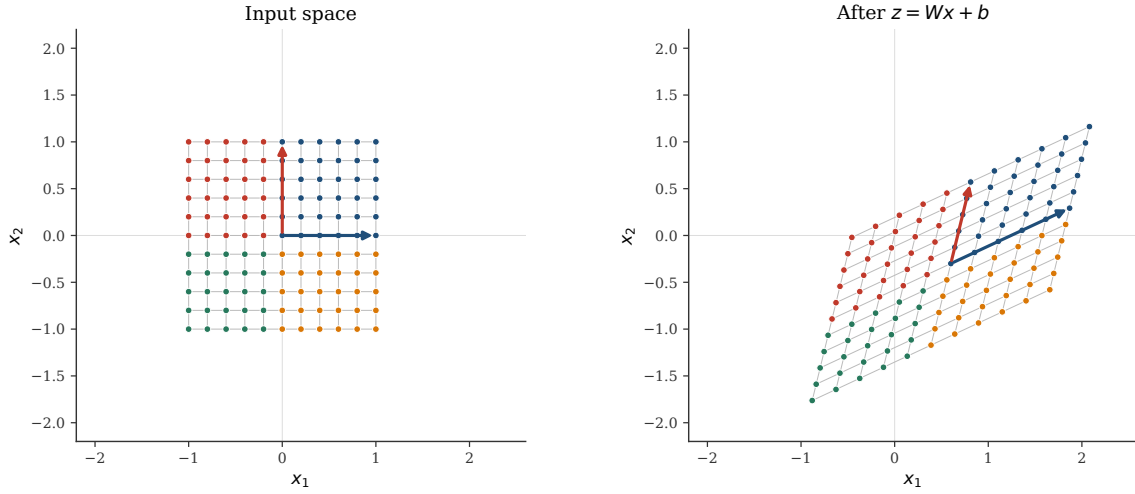


Figure 2: The left panel shows a grid of points in input space. The right panel shows the same grid after applying $z = Wx + b$. The basis vectors e_1 (blue) and e_2 (red) become the columns of W . The bias translates everything so the origin is no longer fixed. Quadrant colors are preserved across the transformation so you can see how the space gets warped.

Batch parallelism. In practice we never feed a network one example at a time. Instead, we stack N inputs as rows of a design matrix $X \in \mathbb{R}^{N \times n}$, and the affine layer becomes

$$Z = XW^\top + B \in \mathbb{R}^{N \times m}, \quad (3)$$

where B broadcasts the bias across rows. This reformulation matters a great deal for performance: dense matrix multiplications (called GEMM, for General Matrix Multiply) are precisely the operation GPUs are designed to execute as massively parallel SIMD workloads, and structuring our computations this way is what lets a single GPU process thousands of examples in the time it would take to process one example badly.

2.2 Why non-linearities are mandatory

Suppose we tried to build a network out of pure affine layers, with nothing between them. Two stacked affine maps would give us

$$W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)} = (W^{(2)}W^{(1)})x + (W^{(2)}b^{(1)} + b^{(2)}) = W_{\text{eq}}x + b_{\text{eq}}. \quad (4)$$

This is just another affine map. No matter how many layers we stack, the whole stack collapses to a single equivalent linear layer. All that depth buys us exactly nothing. The non-linear activations between layers are what bend and fold the space, breaking the collapse and giving compositional depth real expressive power.

Figure 3 makes this concrete. A small trained MLP gradually untangles two interleaved spirals from a non-separable input space into one where a single hyperplane is enough to separate the two classes. Without non-linearities, no amount of depth could do this.

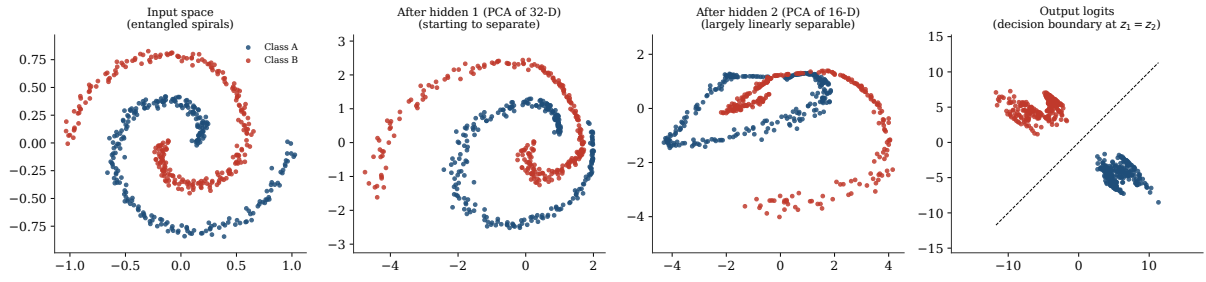


Figure 3: Hierarchical representation learning on a two-spiral classification task. A small MLP ($2 \rightarrow 32 \rightarrow 16 \rightarrow 2$, ReLU activations, trained to 100% accuracy) progressively warps the input manifold. By the time the data reaches the output logits, the two classes are linearly separable. PCA is used to project the high-dimensional hidden activations into 2D for visualization.

2.3 Jacobians and the vector chain rule

Single-variable calculus isn't enough for what we need to do. A layer takes a vector in and returns a vector out, so its derivative isn't a single number — it's a whole matrix of partial derivatives.

Definition 1 (Jacobian). *Let $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ be differentiable. The Jacobian of f at x is the matrix $J_f(x) \in \mathbb{R}^{m \times n}$ with entries*

$$[J_f(x)]_{ij} = \frac{\partial f_i(x)}{\partial x_j}.$$

You can think of the Jacobian as the natural generalization of the derivative: it's a linear map that takes a small perturbation in the input and tells you the corresponding small change in the output.

Two everyday Jacobians. The two Jacobians we'll use again and again are remarkably simple.

- *Affine map.* For $z = Wx + b$, the Jacobian is just $\partial z / \partial x = W$. The map is already linear, so its derivative is itself.
- *Pointwise activation.* For $h = g(z)$ applied elementwise, each h_i depends only on z_i . So $\partial h / \partial z$ is a diagonal matrix: $\text{diag}(g'(z))$.

The vector chain rule. Now suppose $y = f(g(x))$ where $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $f : \mathbb{R}^m \rightarrow \mathbb{R}^k$. The vector chain rule says we can compose Jacobians by multiplying them:

$$J_{f \circ g}(x) = J_f(g(x)) \cdot J_g(x). \quad (5)$$

For the case we actually care about — a scalar loss $\mathcal{L} = \ell(g(x))$ at the end of the network — J_ℓ has shape $1 \times m$, and it's conventional to transpose this into a column gradient vector $\nabla_y \mathcal{L} \in \mathbb{R}^m$. Then Equation (5) becomes:

$$\nabla_x \mathcal{L} = J_g(x)^\top \nabla_y \mathcal{L}. \quad (6)$$

Intuition

Read Equation (6) from right to left and you have the entire idea of backpropagation in one line. We start with the gradient of the loss with respect to the layer's *output*, $\nabla_y \mathcal{L}$. We multiply it by the transposed Jacobian of that layer. The result is the gradient of the loss with respect to the layer's *input*, which is exactly what the previous layer needs. That transpose is the mechanical reason the backward pass moves through the network in reverse order.

2.4 Layout conventions and a worked Jacobian

One source of endless confusion in matrix calculus is layout: should $\partial z / \partial W$ be shaped like W , or like $W^\top$, or like neither? Different textbooks pick different conventions, and the resulting transposes can be maddening to track. Throughout this report we adopt the *denominator layout* (sometimes called the gradient or Hessian layout): the gradient of a scalar with respect to a matrix is shaped exactly like that matrix. This convention has one practical advantage that outweighs all others: a gradient-descent update can be written

$$W \leftarrow W - \eta \nabla_W \mathcal{L}$$

without any reshape or transpose anywhere. This is why every modern deep-learning framework uses it.

Deriving the weight gradient via index notation. Let's actually work out what $\partial \mathcal{L} / \partial W$ should be. Take $z = Wx$ with $z \in \mathbb{R}^m$, $x \in \mathbb{R}^n$, $W \in \mathbb{R}^{m \times n}$, and a downstream scalar loss $\mathcal{L}$. Suppose the backward pass has already given us $\nabla_z \mathcal{L} \in \mathbb{R}^m$; for compactness write its components as $\delta_i = \partial \mathcal{L} / \partial z_i$. The thing we want is $\partial \mathcal{L} / \partial W_{ij}$.

The chain rule plus the fact that $z_k = \sum_\ell W_{k\ell} x_\ell$ gives

$$\frac{\partial \mathcal{L}}{\partial W_{ij}} = \sum_k \frac{\partial \mathcal{L}}{\partial z_k} \frac{\partial z_k}{\partial W_{ij}} = \sum_k \delta_k \cdot \delta_{ki} x_j = \delta_i x_j. \quad (7)$$

Reading the indices as “row i , column j ,” this is exactly the outer product

$$\boxed{\frac{\partial \mathcal{L}}{\partial W} = (\nabla_z \mathcal{L}) x^\top \in \mathbb{R}^{m \times n}.} \quad (8)$$

The bias gradient comes out even simpler: since z_i depends linearly on b_i (and on no other b_j), we just get $\partial \mathcal{L} / \partial b = \nabla_z \mathcal{L}$.

What about the input? The gradient that flows backward into the previous layer is

$$\nabla_x \mathcal{L} = W^\top \nabla_z \mathcal{L} \in \mathbb{R}^n, \quad (9)$$

which is just Equation (6) applied to the affine case.

Equations (8) and (9) are the only two facts you need to backpropagate through a fully-connected layer, and they are worth committing to memory. In the backward pass, a linear layer does two things: a transposed matrix-multiply for the gradient passing through, and an outer product for the weight gradient itself. That's it.

2.5 Batched gradients

When we feed the network a batch $X \in \mathbb{R}^{N \times n}$ rather than a single vector, the per-sample weight gradients $\partial \mathcal{L}^{(i)} / \partial W = (\nabla_z \mathcal{L}^{(i)}) x^{(i)\top}$ are simply summed over the batch:

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{i=1}^N (\nabla_{z^{(i)}} \mathcal{L}) x^{(i)\top} = G^\top X, \quad (10)$$

where $G \in \mathbb{R}^{N \times m}$ has the per-sample upstream gradients stacked as rows. Notice that this is a single dense matrix multiply — the same primitive the forward pass uses. This is why backpropagation is computationally cheap relative to the forward pass: roughly $1\times$ to $2\times$ the FLOPs, not orders of magnitude more.

3 Activation Functions and the Gradient-Flow Problem

In Section 2 we established that depth is only meaningful when affine layers are separated by non-linearities. But *which* non-linearity? This section walks through the standard choices, derives their gradients, and quantifies the central problem that deep saturating activations create: the vanishing gradient.

3.1 The standard menu

Let $g : \mathbb{R} \rightarrow \mathbb{R}$ be applied elementwise to a pre-activation vector z . Here are the activations you'll see most often, along with their derivatives:

Name	Definition $g(z)$	Derivative $g'(z)$	Range
Sigmoid	$\frac{1}{1 + e^{-z}}$	$g(z)(1 - g(z))$, max = 0.25	$(0, 1)$
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - \tanh^2(z)$, max = 1	$(-1, 1)$
ReLU	$\max(0, z)$	$\mathbb{I}[z > 0]$	$[0, \infty)$
Leaky ReLU	$\max(\alpha z, z)$, $\alpha \approx 0.01$	$\mathbb{I}[z > 0] + \alpha \mathbb{I}[z \leq 0]$	$\mathbb{R}$
GELU	$z \cdot \Phi(z)$	smooth, $\approx \text{ReLU}'$ for $ z \gg 0$	$\mathbb{R}$

Table 1: Common activation functions. Φ denotes the standard Gaussian CDF.

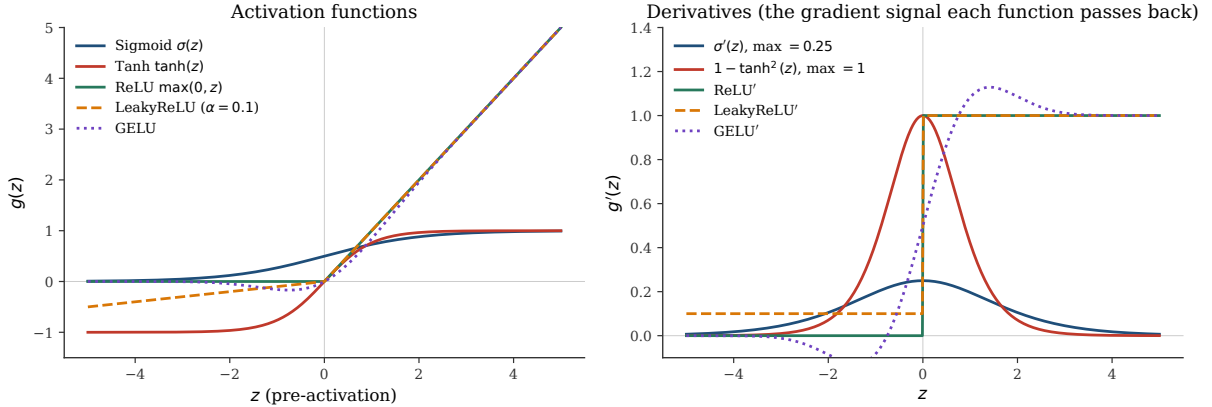


Figure 4: Left: activations side-by-side over $z \in [-5, 5]$. Right: their derivatives, which is what actually multiplies the upstream gradient during backpropagation — so this is the more important plot for understanding training dynamics. Sigmoid’s derivative caps at 0.25 and decays exponentially as $|z|$ grows; tanh peaks at 1 but saturates the same way; ReLU’s derivative is exactly 1 on the active half-line and 0 otherwise; GELU is a smooth ReLU variant that has become standard in transformers.

3.2 The vanishing gradient problem, quantified

Imagine an L -layer network with the same activation g at every layer. By the chain rule, the gradient with respect to a parameter in layer 1 involves a product of L activation derivatives chained together:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} \propto \prod_{l=1}^L g'(z^{(l)}) \cdot \prod_{l=2}^L W^{(l)\top}. \quad (11)$$

Here’s the trouble. If g' is bounded above by some $c < 1$, then this product is bounded by c^L , which shrinks geometrically as the network gets deeper. For the sigmoid, c is exactly 0.25. Even in the most optimistic case for sigmoid, then, the gradient flowing through $L = 30$ saturating layers is bounded by $0.25^{30} \approx 10^{-18}$ — well below the $\sim 10^{-7}$ precision floor of float32. The reality is usually much worse than the worst case: as soon as weight magnitudes are anything but small, z gets pushed into regions where $g'(z)$ is exponentially close to zero, and the gradient is wiped out entirely.

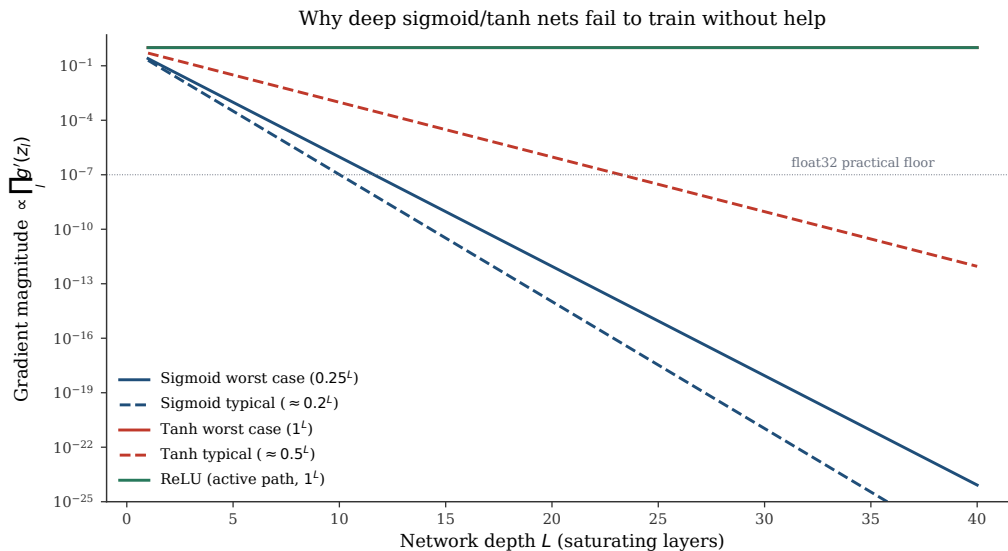


Figure 5: Worst-case and typical gradient magnitudes as a function of network depth, for a network that uses the same activation at every layer. The ReLU active-path gradient is $1^L = 1$ and does not vanish at all (though ReLU has its own problem — see below). Sigmoid saturates the float32 dynamic range within just 30–40 layers; tanh is gentler but eventually does the same thing. This is why neural networks in the early 2000s rarely had more than a handful of trainable layers, and why ReLU, residual connections, and normalization were such pivotal innovations once they arrived.

Key idea

The vanishing gradient is fundamentally a problem with the *product* of derivatives along the backward path. Three families of solutions have shaped modern architectures, and you’ll see all three in any contemporary model:

- (1) Use activations whose derivative stays bounded away from zero on the active region (ReLU, GELU, SiLU).
- (2) Add residual (“skip”) connections so gradients have a shortcut path that bypasses the multiplication through every layer (ResNets, transformers).
- (3) Normalize intermediate activations to keep z in the non-saturating region (BatchNorm, LayerNorm; see Section 8).

3.3 ReLU and its pathologies

ReLU’s flat zero region kills off the vanishing-gradient problem on the active side — the gradient is exactly 1, no decay — but it introduces a complementary problem of its own.

Pitfall

Dying ReLU. Suppose a neuron’s pre-activation z ends up consistently negative for every training input. This can happen after a large weight update, or with a poor initialization. Once it does, $g'(z) = 0$ for every input the network ever sees, and the gradient through that neuron is exactly zero. No gradient means no update, and the neuron is dead forever — it will never recover. Under aggressive learning rates, it’s not unusual for a meaningful fraction of ReLU units to die this way.

Mitigation: Use Leaky ReLU (a small slope $\alpha \approx 0.01$ on the negative side instead of zero), Parametric ReLU (where α is learned), or GELU/SiLU/Swish. All of these have a non-zero gradient everywhere, which prevents permanent death.

3.4 Mean-centered vs. zero-bounded activations

Sigmoid outputs live in $(0,1)$ — always positive — and ReLU outputs live in $[0,\infty)$. This positive bias has a subtle but important consequence as it propagates through the network. When all the incoming activations to a layer have the same sign, every entry of the weight gradient $\partial\mathcal{L}/\partial W = (\nabla_z\mathcal{L})h^\top$ from Equation (8) ends up with the same sign as $\nabla_z\mathcal{L}$. The weight update then moves all the weights in a row in the same direction at the same time, producing the characteristic zig-zag descent that LeCun warned about back in the 1990s.

Tanh, GELU, and SiLU are all roughly zero-centered and avoid this problem. This is one of the historical reasons tanh outperformed sigmoid in hidden layers, and one of the reasons GELU has become the dominant choice in modern transformers.

3.5 What about output activations?

Everything above concerns *hidden* layers. The output layer is a different story, because what it does is constrained by what the loss expects to see:

- **Regression:** identity (no activation), so the output is unbounded in $\mathbb{R}^k$.
- **Binary classification:** sigmoid, producing a probability in $(0,1)$.
- **Multi-class classification:** softmax (Section 4), producing a probability vector in the simplex.
- **Bounded outputs** (e.g., image generation into $[-1,1]$): tanh.

The hidden-layer pathologies we just discussed largely don’t apply to the output layer, because that layer only contributes a single Jacobian to the backward chain, not L of them stacked up.

4 Loss Functions: Quantifying Disagreement

A loss function $\mathcal{L}(\hat{y}, y)$ is a scalar that measures how badly the network’s prediction $\hat{y}$ disagrees with the ground truth y . The right choice depends on the task — regression, binary classification, or multi-class classification — and it ends up shaping both the output activation we use and the gradient that flows back into the network when we train.

4.1 Mean squared error for regression

For a regression target $y \in \mathbb{R}^k$, the workhorse loss is mean squared error (often written L2 loss):

$$\mathcal{L}_{\text{MSE}}(\hat{y}, y) = \frac{1}{2} \|\hat{y} - y\|_2^2 = \frac{1}{2} \sum_{i=1}^k (\hat{y}_i - y_i)^2. \quad (12)$$

The factor of $\frac{1}{2}$ has no statistical meaning — it’s there purely to cancel the factor of 2 that would appear when we differentiate the square. The gradient is then

$$\nabla_{\hat{y}} \mathcal{L}_{\text{MSE}} = \hat{y} - y \in \mathbb{R}^k, \quad (13)$$

which is just the residual: the signed error in each coordinate. From a probabilistic perspective, minimizing MSE corresponds to maximum-likelihood estimation under the assumption that y is Gaussian-distributed around $\hat{y}$ with a fixed variance.

4.2 Cross-entropy for classification

For classification, the network’s job is to produce a probability vector $\hat{p} \in \mathbb{R}^K$ over K classes. The target y is the true class index (or its one-hot vector representation). The cross-entropy between the true and predicted distributions is

$$\mathcal{L}_{\text{CE}}(\hat{p}, y) = - \sum_{k=1}^K y_k \log \hat{p}_k = -\log \hat{p}_{y^*}, \quad (14)$$

where y^* is the index of the true class. Information-theoretically, this is the “surprise” the model assigns to the correct answer: high if the model thought the right answer was unlikely, low if it was confident.

Why not just use MSE for classification? Two reasons, both visible in Figure 6.

1. *Asymmetric penalty.* Cross-entropy goes to $+\infty$ as $\hat{p}_{y^*} \rightarrow 0$: being confidently wrong is infinitely costly. MSE just saturates — a confidently wrong sigmoid output gives an MSE near 1, regardless of how wildly off the prediction was.
2. *Gradient signal.* A sigmoid followed by MSE has a gradient proportional to $\sigma'(z)$, which vanishes when the model is confidently wrong. That’s the worst possible case — exactly when you’d want a large corrective signal, you get a tiny one. Softmax with cross-entropy, on the other hand, gives a gradient proportional to $\hat{p} - y$ (we’ll derive this below). That gradient is *largest* precisely when the model is most wrong, which is exactly the property a well-behaved loss should have.

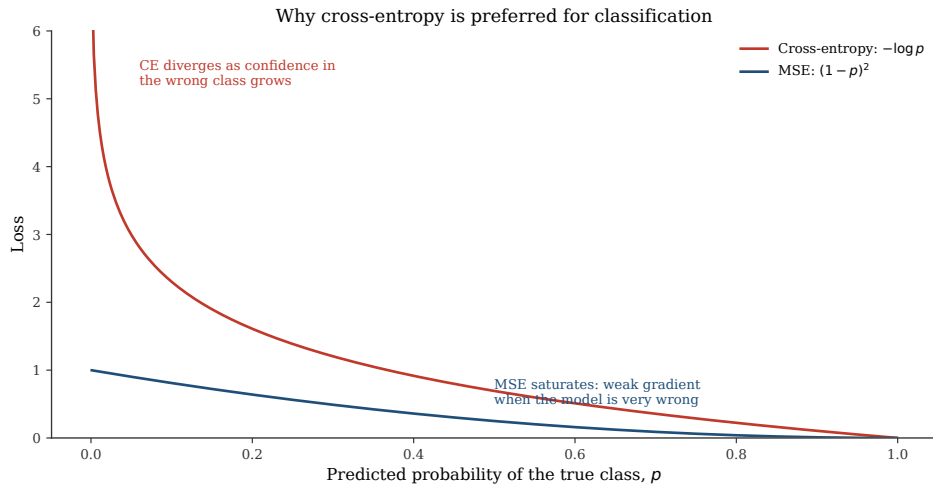


Figure 6: Loss as a function of p , the probability the model assigns to the true class. Cross-entropy punishes confident mistakes severely and rewards confident correctness sharply. MSE flattens out, giving weak training signal at exactly the wrong moment — when the model is most confidently wrong.

4.3 Softmax: turning logits into probabilities

Cross-entropy needs a probability vector as input, but the network’s last linear layer produces unbounded *logits* $z \in \mathbb{R}^K$. The softmax function is what bridges this gap, mapping any logit vector to a point on the probability simplex:

$$\hat{p}_k = \text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}. \quad (15)$$

By construction, every $\hat{p}_k > 0$ and they sum to 1. Softmax also has a useful shift-invariance property: $\text{softmax}(z) = \text{softmax}(z + c\mathbf{1})$ for any constant c . We can subtract anything we want from every logit and get the same answer. This is the basis of the log-sum-exp trick for numerical stability:

```
def softmax(z):
    z = z - z.max(axis=-1, keepdims=True)  # log-sum-exp trick
    e = np.exp(z)
    return e / e.sum(axis=-1, keepdims=True)
```

Subtracting the max guarantees that the largest exponent we compute is $e^0 = 1$, which prevents overflow when one of the logits is large (which is exactly what we want to happen in a confident model).

4.4 The softmax–cross-entropy gradient

Here’s a result that is, honestly, one of the most beautiful in deep learning: when you compose softmax with cross-entropy, the gradient comes out perfectly clean — no special functions, no exponentials surviving in the answer, no K -way summation. Let y be the one-hot vector for the true class and z the logits. Then with $\hat{p} = \text{softmax}(z)$:

$$\nabla_z \mathcal{L}_{\text{CE}} = \hat{p} - y. \quad (16)$$

Derivation. Start from $\mathcal{L} = -\log \hat{p}_{y^*} = -z_{y^*} + \log \sum_j e^{z_j}$. Now differentiate with respect to z_k :

$$\frac{\partial \mathcal{L}}{\partial z_k} = -\mathbb{1}[k = y^*] + \frac{e^{z_k}}{\sum_j e^{z_j}} = -y_k + \hat{p}_k = \hat{p}_k - y_k.$$

Stacked over all k , this is exactly Equation (16).

Intuition

The gradient $\hat{p}-y$ has a wonderfully clean interpretation: it's just "the predicted distribution minus the true distribution." It's large in coordinates where the model is wrong and small in coordinates where it's right. The magnitude per coordinate tells you exactly how much probability mass needs to be moved. This is why frameworks fuse softmax and cross-entropy into a single op (`F.cross_entropy` in PyTorch, `softmax_cross_entropy_with_logits` in TensorFlow): the fused version is more numerically stable than the two-step version, and it's trivially differentiable thanks to this elegant gradient.

4.5 Other losses worth knowing

A few other losses come up often enough to mention:

- **Binary cross-entropy** (for a single sigmoid output): $\mathcal{L} = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$. The gradient with respect to the logit is again $\hat{p} - y$ — the same clean form as the multi-class case.
- **Huber loss** (for robust regression): quadratic for small residuals and linear for large ones. This combines MSE's smoothness near the optimum with L1's robustness to outliers.
- **Hinge loss** (max-margin classification, SVM-style): $\mathcal{L} = \max(0, 1 - y\hat{y})$. Not technically differentiable at the margin, but subgradients work fine. Less common in modern deep learning except in specific structured-output settings.
- **Negative log-likelihood for distributions**: a generalization of cross-entropy. If the network outputs the parameters $\theta_\phi(x)$ of some distribution p_θ , the loss is just $-\log p_{\theta_\phi(x)}(y)$. This formulation covers a huge swath of modern generative modelling, including the diffusion-model objective.

5 Optimization: From Gradient Descent to Adam

Once we can compute $\nabla_{\theta}\mathcal{L}$, the next question is what to actually do with those gradients. The answer is gradient-based optimization, and modern deep learning has built up a sizeable toolbox of variants on the basic gradient-descent recipe. This section traces the lineage from full-batch GD all the way to today’s de facto default: Adam.

5.1 Gradient descent on a non-convex landscape

The basic update rule is the one everyone has seen:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta}\mathcal{L}(\theta^{(t)}), \quad (17)$$

where $\eta > 0$ is the learning rate. For a convex loss, with a small enough step size, this provably converges to the global minimum. For the wildly non-convex loss landscape of a deep network — full of saddle points, plateaus, and local minima — no such guarantee exists. And yet, empirically, gradient descent works remarkably well. Modern theory partly explains why: in high dimensions, most critical points turn out to be saddles rather than minima, and the local minima we do encounter tend to have loss values comparable to the global minimum. So while the picture in low-dimensional cartoons looks scary, the actual optimization problem is friendlier than it sounds.

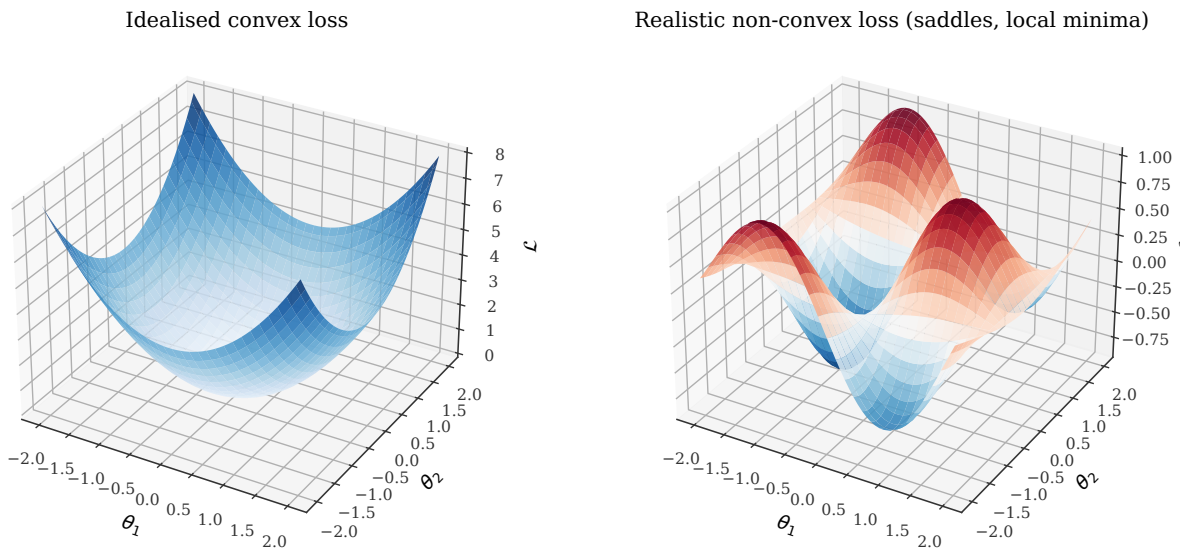


Figure 7: Left: an idealized convex loss — the picture from textbooks. Right: a more realistic non-convex surface with multiple minima and saddle points. The actual neural-network loss is a function on $\mathbb{R}^d$ with d in the millions or billions, so this 2D slice is at best a metaphor. But it captures the qualitative reason optimization is non-trivial.

5.2 Stochastic gradient descent

The exact gradient $\nabla\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla\mathcal{L}_i(\theta)$ averages over the entire training set. When N is in the millions or billions — as it always is for modern training — computing this even once is prohibitively expensive, let alone at every step. The workaround is stochastic gradient descent (SGD): replace the exact gradient with an unbiased estimate computed on a randomly drawn mini-batch $\mathcal{B}_t$ of size $B \ll N$:

$$\tilde{\nabla}\mathcal{L}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla\mathcal{L}_i(\theta). \quad (18)$$

This estimator is unbiased — on average it equals the true gradient, $\mathbb{E}[\tilde{\nabla}\mathcal{L}] = \nabla\mathcal{L}$ — and its variance scales as $\mathcal{O}(1/B)$, so larger batches give us less noisy estimates.

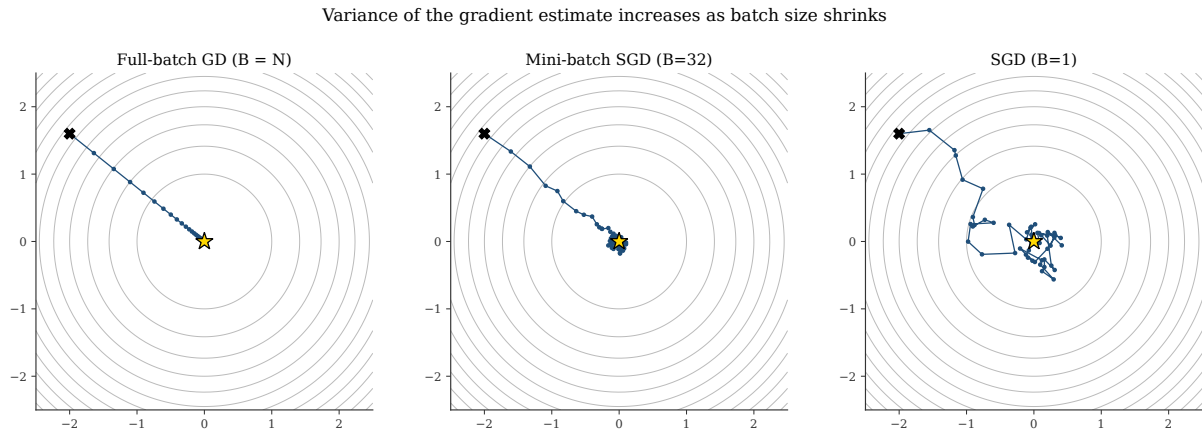


Figure 8: Three optimizer trajectories from the same starting point on the same quadratic loss. Full-batch gradient descent (left) follows the deterministic steepest-descent path. Mini-batch SGD with $B = 32$ (middle) wobbles a bit but converges efficiently. Per-example SGD with $B = 1$ (right) is much noisier, but it still converges — and that noise can actually help the model escape sharp local minima.

What’s the right batch size? One of the more surprising findings of the past decade is that batch size matters in ways that go well beyond the obvious time-per-step tradeoff. Larger batches give lower-variance gradients but tend to generalize worse for the same number of epochs — this is the so-called “generalization gap.” Smaller batches act as a kind of implicit regularizer but need more iterations to converge. In practice you’ll see batch sizes of 32–256 for image models and 1M+ tokens for large language models. There’s even a research subfield devoted to the relationship between batch size and effective learning rate (the “linear scaling rule” and its various refinements).

5.3 Momentum

Stochastic gradients are noisy by construction, and along directions of high curvature they oscillate back and forth instead of making progress. *Momentum* [8] smooths the trajectory by keeping an exponentially weighted moving average of past gradients:

$$v^{(t)} = \mu v^{(t-1)} + \nabla\mathcal{L}(\theta^{(t)}), \quad (19)$$

$$\theta^{(t+1)} = \theta^{(t)} - \eta v^{(t)}. \quad (20)$$

The hyperparameter $\mu \in [0, 1]$ — typically 0.9 — controls how much past gradient information we retain. Setting $\mu = 0$ recovers vanilla SGD.

Intuition

The physical analogy is genuinely useful here: imagine a heavy ball rolling on the loss surface. The ball accumulates velocity in consistent directions, which lets it traverse flat plateaus and shallow saddles that would stall pure gradient descent. In oscillating directions, the opposing gradients cancel out in the moving average, so the ball doesn’t drift sideways — it just rolls downhill.

5.4 Adaptive learning rates: RMSProp and Adam

Here’s a useful observation: different parameters in a network often want very different learning rates. A parameter that consistently sees huge gradients needs small steps; one that consistently sees tiny gradients needs amplification. Adaptive optimizers track per-parameter statistics of the gradient and scale each step accordingly.

RMSProp [9]. The idea is to keep a running average of squared gradients, then divide each step by its square root:

$$s^{(t)} = \beta s^{(t-1)} + (1 - \beta) g^{(t)} \odot g^{(t)}, \quad (21)$$

$$\theta^{(t+1)} = \theta^{(t)} - \frac{\eta}{\sqrt{s^{(t)}} + \varepsilon} g^{(t)}. \quad (22)$$

Parameters with consistently large gradients end up with small effective learning rates, and rare parameters with tiny gradients get amplified. This is enormously helpful when gradient magnitudes vary across coordinates by many orders of magnitude.

Adam [10]. Adam combines momentum (first moment) with RMSProp (second moment), and throws in a bias correction to fix the fact that the moving averages start at zero:

$$m^{(t)} = \beta_1 m^{(t-1)} + (1 - \beta_1) g^{(t)}, \quad (23)$$

$$v^{(t)} = \beta_2 v^{(t-1)} + (1 - \beta_2) g^{(t)} \odot g^{(t)}, \quad (24)$$

$$\hat{m}^{(t)} = m^{(t)} / (1 - \beta_1^t), \quad \hat{v}^{(t)} = v^{(t)} / (1 - \beta_2^t), \quad (25)$$

$$\theta^{(t+1)} = \theta^{(t)} - \frac{\eta}{\sqrt{\hat{v}^{(t)}} + \varepsilon} \hat{m}^{(t)}. \quad (26)$$

The default hyperparameters $\beta_1 = 0.9, \beta_2 = 0.999, \varepsilon = 10^{-8}$ work astonishingly well across an enormous range of architectures and datasets — which is the main reason Adam (or its variant AdamW, which decouples weight decay from the gradient [11]) is the default optimizer in essentially all modern deep-learning code.

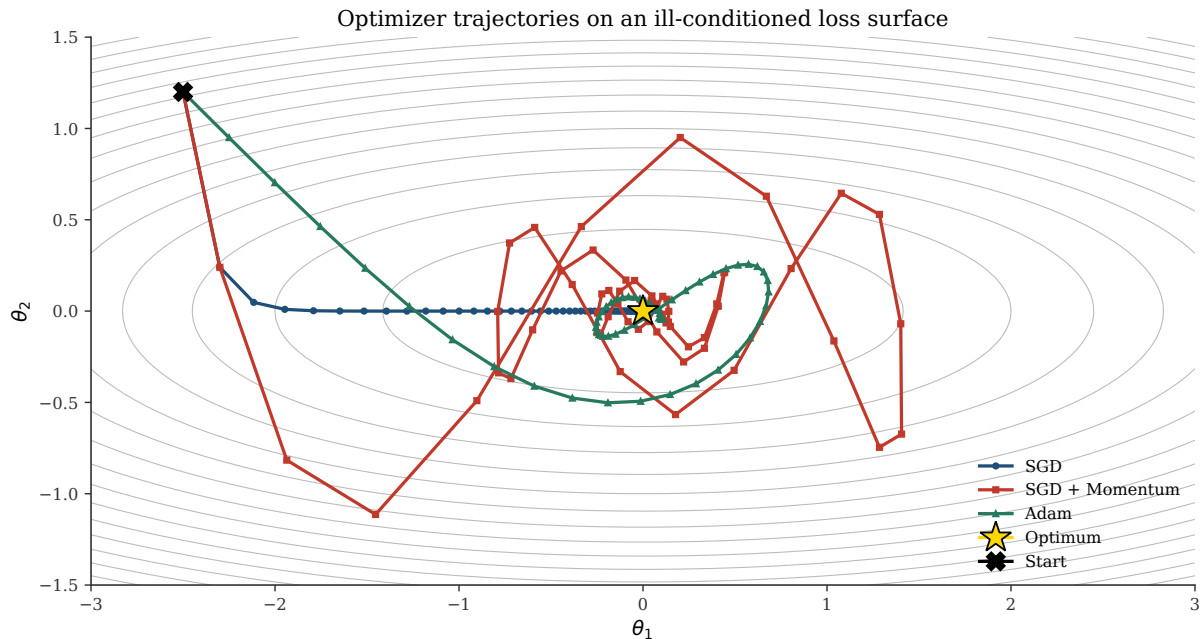


Figure 9: Three optimizers tackling an ill-conditioned quadratic loss (vertical curvature $10\times$ horizontal). SGD with a step size small enough to be safe in the vertical direction can barely make horizontal progress. Momentum accelerates the horizontal motion but ends up overshooting and oscillating. Adam’s per-parameter adaptation handles the conditioning automatically: it takes large effective steps in the flat direction and small ones in the curved direction.

5.5 Learning-rate schedules

A single fixed learning rate is rarely optimal throughout training. The common schedules you’ll see are:

- **Step decay:** multiply η by 0.1 every K epochs. Crude but surprisingly effective.
- **Cosine annealing** [12]: smoothly decay η from a maximum down to a minimum following a cosine curve. This has become the default for most modern training runs.
- **Warmup:** linearly ramp η up from 0 to its target over the first few thousand steps. This is especially important for transformers, where the uninitialized attention weights produce wildly variable gradients in the first few steps.
- **One-cycle / triangular schedules** [13]: increase η then decrease it within a single epoch, sometimes producing super-convergence to a good minimum in far fewer steps than expected.

The cocktail of *linear warmup followed by cosine decay* is, at the moment, the de facto modern default for training large models.

5.6 Pseudocode summary

Algorithm 1 Adam (Kingma & Ba, 2014)

Require: learning rate η , decays β_1, β_2 , ε , initial θ

```
1:  $m \leftarrow 0, v \leftarrow 0, t \leftarrow 0$ 
2: while not converged do
3:    $t \leftarrow t + 1$ 
4:   sample mini-batch and compute  $g \leftarrow \nabla_{\theta} \mathcal{L}(\theta)$ 
5:    $m \leftarrow \beta_1 m + (1 - \beta_1) g$ 
6:    $v \leftarrow \beta_2 v + (1 - \beta_2) g \odot g$ 
7:    $\hat{m} \leftarrow m / (1 - \beta_1^t)$ 
8:    $\hat{v} \leftarrow v / (1 - \beta_2^t)$ 
9:    $\theta \leftarrow \theta - \eta \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$ 
10: end while
```

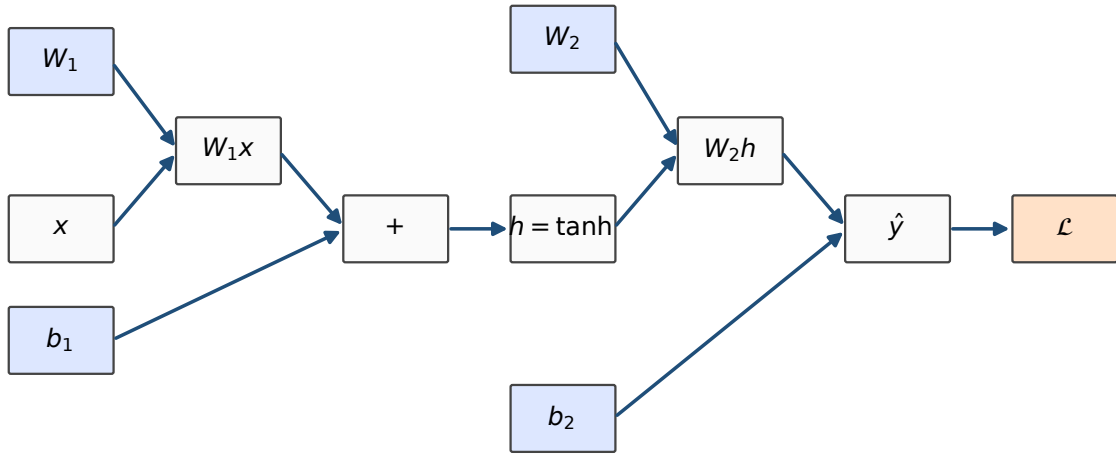
6 The Training Protocol: Forward and Backward Pass

Now it's time to assemble the pieces into the full training loop. Each iteration of training has three logical phases: a forward pass that computes activations and loss, a backward pass that computes gradients via reverse-mode automatic differentiation, and an optimizer step that uses those gradients to update parameters.

6.1 The two passes through the computation graph

The forward pass evaluates the network by walking the computation graph in topological order, caching every intermediate tensor that the backward pass will need to see. The backward pass then walks the same graph in reverse, applying the local Jacobian-vector products of Equations (8) and (9) at each node.

Forward pass: cache every intermediate activation in memory



Backward pass: chain-rule gradients flow right-to-left

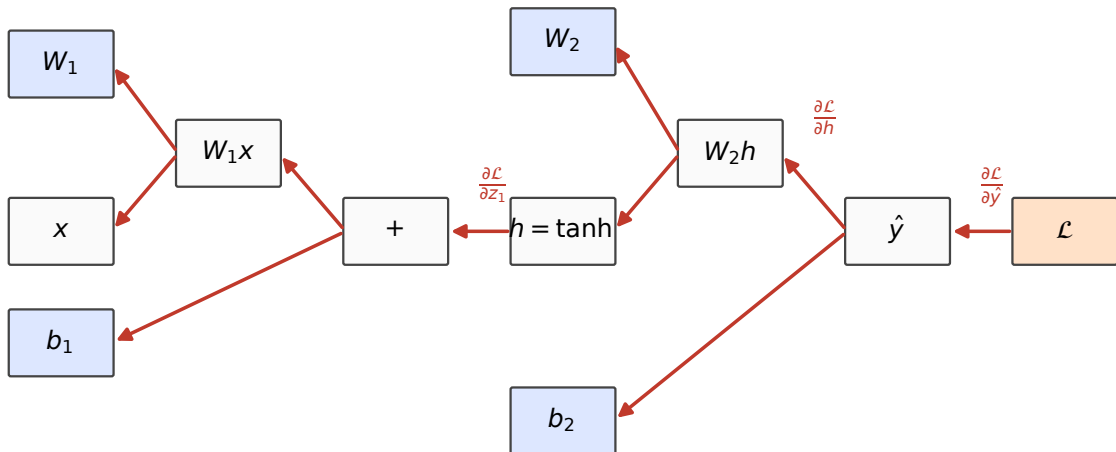


Figure 10: The same two-layer computation graph shown in forward form (top, blue arrows) and backward form (bottom, red arrows). The forward pass computes $\hat{y} = W_2 \tanh(W_1x + b_1) + b_2$ along with the loss $\mathcal{L}$, caching every intermediate tensor as it goes. The backward pass then propagates $\partial \mathcal{L} / \partial \hat{y}$ leftward through the graph, applying local Jacobian rules at each node to recover gradients for every cached value and every parameter.

Why cache? The local gradient of every operation depends on its inputs, and sometimes on its output too. To pick two examples: the gradient through a ReLU needs to know whether each pre-activation was positive, and the gradient through a softmax needs the softmax output. The framework therefore has to hold onto every intermediate tensor in memory until the backward pass consumes it. This is the dominant memory cost of training, and it's the main reason inference uses far less memory than training (we'll return to this point in Section 10).

6.2 Worked example: backpropagation by hand

Let's walk through a complete forward and backward pass on a tiny two-layer regression network, being careful with shapes and conventions at every step. This is the same example structure that appears in standard introductory treatments, but we'll do it with the corrected, consistently-laid-out gradient computations developed in Section 2.

Setup. The network is

$$\hat{y} = W_1 \tanh(W_0 x).$$

No biases, for brevity. Concretely:

$$x = \begin{pmatrix} 1.0 \\ 0.1 \end{pmatrix} \in \mathbb{R}^2, \quad y = 0.5 \in \mathbb{R},$$

$$W_0 = \begin{pmatrix} 1 & -3 \\ 0.2 & 1 \end{pmatrix} \in \mathbb{R}^{2 \times 2}, \quad W_1 = \begin{pmatrix} 1 & -1 \end{pmatrix} \in \mathbb{R}^{1 \times 2}.$$

The loss is $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2 \in \mathbb{R}$.

Forward pass.

$$z_1 = W_0 x = \begin{pmatrix} 1 \cdot 1.0 + (-3) \cdot 0.1 \\ 0.2 \cdot 1.0 + 1 \cdot 0.1 \end{pmatrix} = \begin{pmatrix} 0.7 \\ 0.3 \end{pmatrix} \in \mathbb{R}^2,$$

$$h_1 = \tanh(z_1) = \begin{pmatrix} \tanh(0.7) \\ \tanh(0.3) \end{pmatrix} \approx \begin{pmatrix} 0.6044 \\ 0.2913 \end{pmatrix} \in \mathbb{R}^2,$$

$$\hat{y} = W_1 h_1 = 1 \cdot 0.6044 + (-1) \cdot 0.2913 \approx 0.3131 \in \mathbb{R},$$

$$\mathcal{L} = \frac{1}{2}(0.3131 - 0.5)^2 \approx 0.01746.$$

Cached for the backward pass: $x, z_1, h_1, \hat{y}$.

Backward pass. We work right-to-left through the graph, maintaining the convention that gradients have the same shape as the quantities they are gradients of.

(1) *Gradient at the output.* From $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y \approx -0.1869 \in \mathbb{R}.$$

(2) *Gradient w.r.t. W_1 .* Using Equation (8) with the scalar upstream gradient -0.1869 and the cached h_1 :

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} h_1^\top \approx (-0.1869) \begin{pmatrix} 0.6044 & 0.2913 \end{pmatrix} = \begin{pmatrix} -0.1130 & -0.0544 \end{pmatrix} \in \mathbb{R}^{1 \times 2}.$$

Note the shape: matches $W_1 \in \mathbb{R}^{1 \times 2}$.

(3) *Gradient w.r.t. h_1 .* Using Equation (9):

$$\frac{\partial \mathcal{L}}{\partial h_1} = W_1^\top \frac{\partial \mathcal{L}}{\partial \hat{y}} \approx \begin{pmatrix} 1 \\ -1 \end{pmatrix} (-0.1869) = \begin{pmatrix} -0.1869 \\ 0.1869 \end{pmatrix} \in \mathbb{R}^2.$$

(4) *Gradient through the tanh.* The Jacobian of the elementwise tanh is $\text{diag}(1 - \tanh^2(z_1))$, so backpropagating is an elementwise product (denoted $\odot$):

$$\frac{\partial \mathcal{L}}{\partial z_1} = (1 - h_1 \odot h_1) \odot \frac{\partial \mathcal{L}}{\partial h_1}.$$

Compute $1 - h_1 \odot h_1 \approx (1 - 0.3653, 1 - 0.0849) = (0.6347, 0.9151)$:

$$\frac{\partial \mathcal{L}}{\partial z_1} \approx \begin{pmatrix} 0.6347 \\ 0.9151 \end{pmatrix} \odot \begin{pmatrix} -0.1869 \\ 0.1869 \end{pmatrix} = \begin{pmatrix} -0.1186 \\ 0.1710 \end{pmatrix} \in \mathbb{R}^2.$$

(5) *Gradient w.r.t. W_0 .* Apply Equation (8) one more time, with upstream $\partial \mathcal{L} / \partial z_1$ and cached x :

$$\frac{\partial \mathcal{L}}{\partial W_0} = \frac{\partial \mathcal{L}}{\partial z_1} x^\top \approx \begin{pmatrix} -0.1186 \\ 0.1710 \end{pmatrix} \begin{pmatrix} 1.0 & 0.1 \end{pmatrix} = \begin{pmatrix} -0.1186 & -0.01186 \\ 0.1710 & 0.01710 \end{pmatrix} \in \mathbb{R}^{2 \times 2}.$$

Shape matches W_0 , as required.

Parameter update. With learning rate $\eta = 0.1$:

$$W_1 \leftarrow W_1 - 0.1 \frac{\partial \mathcal{L}}{\partial W_1} \approx \begin{pmatrix} 1.0113 & -0.9946 \end{pmatrix}, \quad W_0 \leftarrow W_0 - 0.1 \frac{\partial \mathcal{L}}{\partial W_0}.$$

After this single step, recomputing the forward pass gives $\hat{y} \approx 0.331$ — closer to $y = 0.5$ than the original 0.313.

Key idea

Shape sanity check. In a correctly written backward pass, every gradient tensor has the same shape as the tensor it is a gradient of. If $W_0 \in \mathbb{R}^{m \times n}$ then $\partial \mathcal{L} / \partial W_0 \in \mathbb{R}^{m \times n}$. This makes the optimizer update $\theta \leftarrow \theta - \eta \nabla \mathcal{L}$ a pure elementwise operation with no reshapes. Shape mismatches in handwritten backprop are almost always a transpose error.

6.3 Automatic differentiation, in one paragraph

Modern frameworks (PyTorch, JAX, TensorFlow) don't make you write any of this by hand. Every forward operation quietly records a small piece of metadata: which inputs it consumed, and which local Jacobian-vector-product function to call when gradients arrive from above. The collection of these records forms a runtime computation graph. When you call `loss.backward()`, the framework walks this graph in reverse topological order, invokes each recorded JVP function in turn with the gradient flowing in, and accumulates the result into the appropriate leaf tensor's `.grad` attribute. This is reverse-mode automatic differentiation, and it is exactly the procedure we just performed by hand — except automated, well-tested, and JIT-compiled.

7 Initialization: Where Training Begins

Where you start in parameter space matters enormously. Poorly initialized networks suffer from either vanishing or exploding activations within just a few layers, and once that happens, the optimizer simply cannot recover. This section quantifies the variance dynamics across depth and walks through the two initialization schemes — Xavier and He — that solved this problem and made the modern era of deep networks possible.

7.1 Why all-zeros doesn't work

The most natural-sounding initialization, “set every weight to zero,” is catastrophic. With $W^{(l)} = 0$, every neuron in a hidden layer computes the exact same pre-activation, receives the exact same gradient, and updates identically. The network never breaks the symmetry between neurons, so they all remain copies of each other forever, no matter how long you train. We need random initialization just to break this symmetry.

7.2 Variance preservation across layers

The guiding principle behind modern initialization is simple: we want to preserve the variance of activations as the signal moves forward through the network, and the variance of gradients as the signal moves backward. Concretely, we want $\text{Var}[a^{(l+1)}] \approx \text{Var}[a^{(l)}]$ at every layer.

Forward analysis. Consider an affine layer $z = Wx$ where $W \in \mathbb{R}^{m \times n}$ has i.i.d. entries with zero mean and variance σ_W^2 , and x has i.i.d. entries with variance σ_x^2 . Each output $z_i = \sum_{j=1}^n W_{ij}x_j$ then has variance

$$\text{Var}[z_i] = \sum_{j=1}^n \text{Var}[W_{ij}x_j] = n \sigma_W^2 \sigma_x^2. \quad (27)$$

For $\text{Var}[z] \approx \text{Var}[x]$ to hold, we need $\sigma_W^2 = 1/n$, where n is the layer's *fan-in* — the number of inputs feeding into each neuron.

7.3 Xavier (Glorot) initialization

For activations that are roughly linear near the origin — tanh, identity — Glorot and Bengio [6] proposed sampling weights from a distribution with variance

$$\sigma_W^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}. \quad (28)$$

This is a compromise between forward variance preservation (which wants $1/n_{\text{in}}$) and backward variance preservation (which wants $1/n_{\text{out}}$). In practice, weights are drawn either from $\mathcal{N}(0, 2/(n_{\text{in}} + n_{\text{out}}))$ or from a uniform distribution with matching variance.

7.4 He initialization

ReLU breaks Xavier's assumption, because ReLU zeros out roughly half of its inputs — which halves the variance at every layer. He et al. [7] corrected for this with

$$\sigma_W^2 = \frac{2}{n_{\text{in}}}, \quad (29)$$

where the factor of 2 compensates for the halving caused by ReLU. He initialization (also called Kaiming initialization, after the first author) is the default in PyTorch's `torch.nn.Linear` and `Conv2d` layers.

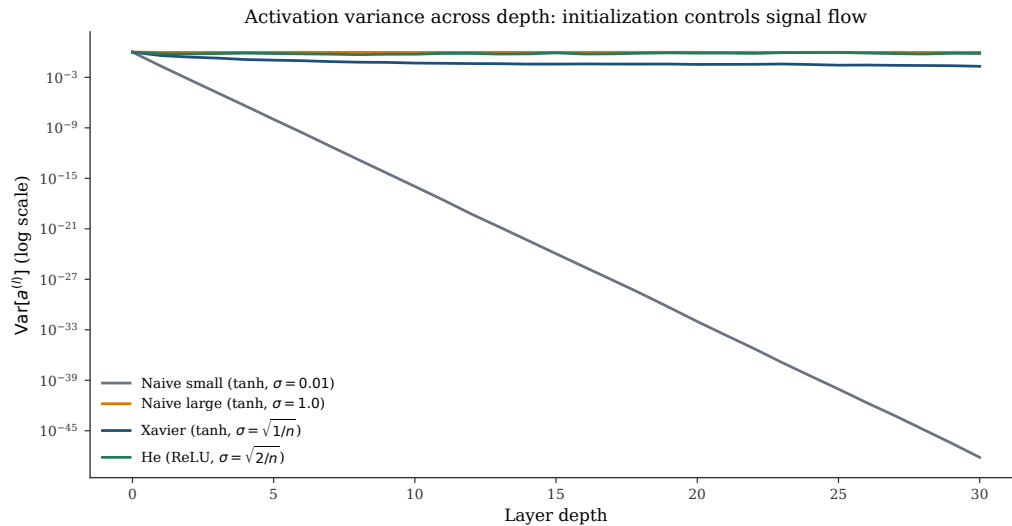


Figure 11: Activation variance as a function of depth, for the same 30-layer fully-connected architecture under four different initialization schemes. Naive small initialization causes the signal to vanish exponentially. Naive large initialization causes it to explode in the other direction. Xavier (for tanh) and He (for ReLU) both maintain approximately constant variance across all 30 layers — which is what allows training to actually proceed.

Intuition

Initialization isn't a footnote. It is, quite literally, the difference between a network that trains and one that produces NaNs. The plot above is part of why deep networks didn't really exist before 2010: without principled initialization, the gradient signal in a 30-layer tanh network was either lost in numerical noise or saturated within the first few epochs, and there was no way to recover.

8 Regularization: Fighting Overfitting

A deep network usually has more than enough capacity to memorize its training set — often quite literally, especially when the dataset is small. Memorization at the cost of generalization is the classical failure mode of overfitting. Regularization is the umbrella term for the collection of techniques we use to nudge the model toward simpler, more generalizable solutions.

8.1 L2 weight decay

The simplest regularizer is to add a penalty on the squared L2 norm of the weights directly to the loss:

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \frac{\lambda}{2} \|\theta\|_2^2. \quad (30)$$

In the gradient update, this is equivalent to shrinking the weights toward zero at every step:

$$\theta^{(t+1)} = (1 - \eta\lambda) \theta^{(t)} - \eta \nabla_{\theta} \mathcal{L}(\theta^{(t)}). \quad (31)$$

From a Bayesian perspective this corresponds to placing a Gaussian prior on θ centered at zero. From a geometric perspective it shrinks the radius of the parameter sphere we’re willing to consider. Typical values of λ are in the range 10^{-4} to 10^{-2} .

AdamW vs. Adam-with-L2. A subtle point: just combining vanilla Adam with an L2 term in the loss does *not* actually give you “Adam with weight decay.” Adam’s adaptive denominator $\sqrt{\hat{v}} + \varepsilon$ rescales the weight-decay term, weakening it for parameters that have seen large gradients historically. AdamW [11] decouples the two: it applies the weight decay directly to the parameters rather than routing it through the gradient. AdamW is now the default for transformer training, and the difference matters.

8.2 Dropout

Dropout [14] stochastically zeros out each activation independently with probability p at training time, then rescales the surviving activations by $1/(1 - p)$ to keep the expected value the same:

$$\tilde{a}_i = \begin{cases} a_i/(1 - p) & \text{with probability } 1 - p, \\ 0 & \text{with probability } p. \end{cases} \quad (32)$$

At inference time, dropout is disabled and the full network is used. The effect during training is to implicitly train an exponentially large ensemble of subnetworks that share parameters, with each minibatch sampling a different subnetwork. Because no single neuron can ever be relied upon (it might be dropped out next time), the network is forced to learn redundant, distributed features. Empirically this reduces overfitting.

Dropout: stochastic node masking creates implicit ensembling

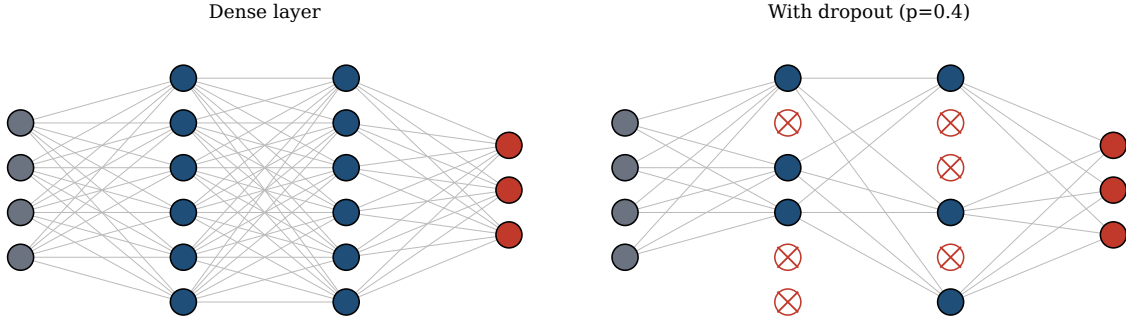


Figure 12: Dropout in action. Left: a normal fully-connected layer. Right: the same layer with $p = 0.4$ dropout applied to the hidden units. The crossed-out neurons are temporarily removed from the network — all their incoming and outgoing connections are inactive for this minibatch. A different random subset gets dropped each minibatch.

When to use dropout. Dropout is most effective in the fully-connected layers of small-to-medium models, especially on small datasets. In transformer architectures it's commonly applied to attention weights and feed-forward outputs. As models and datasets have grown larger, dropout's impact has diminished, and many recent large models use it sparingly or not at all.

8.3 Batch normalization

Batch normalization [15] works by computing, for each minibatch, the mean and variance of each pre-activation across the batch dimension, then using them to normalize:

$$\hat{z}_i = \frac{z_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad \tilde{z}_i = \gamma \hat{z}_i + \beta. \quad (33)$$

The learned per-channel parameters γ and β give the network the freedom to undo the normalization if it ever wanted to. In practice it almost never does, but having those parameters there adds stability and expressive power. At inference time, μ and σ^2 are replaced by running averages that were accumulated during training.

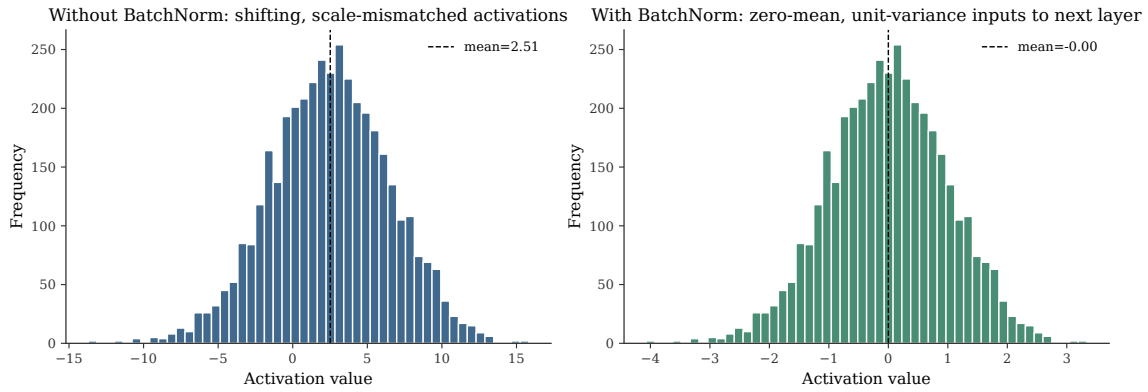


Figure 13: The effect of batch normalization on a pre-activation distribution. Without BN (left), activations entering the next layer can be off-center and badly scaled — exactly the regime where saturating activations vanish. With BN (right), the input to each layer is zero-mean and unit-variance, which preserves signal across depth and keeps everything in the well-behaved part of any activation function.

Why does BN work? The original paper [15] argued that BN reduces “internal covariate shift” — the idea that the distribution of inputs to deep layers keeps changing as earlier layers update, which makes training harder. Later analysis [16] suggested that the dominant effect is actually smoothing the loss landscape, which makes gradients more predictable and lets you use much larger learning rates safely. Whatever the precise mechanism is, BN reliably accelerates training and acts as a mild regularizer (the per-batch statistics inject stochasticity comparable to dropout).

Variants. LayerNorm normalizes across the feature dimension within each example rather than across the batch — which is crucial for sequence models where batch statistics aren’t reliable. GroupNorm and InstanceNorm are intermediate variants used heavily in vision and generative modelling respectively. Transformers use LayerNorm essentially everywhere.

8.4 Early stopping

One of the simplest regularizers — and one of the most effective — is just to monitor validation loss during training and stop when it starts going up, then revert to the best checkpoint. This is implicit regularization: it limits the effective capacity of the model to whatever can be learned within the allowed number of epochs.

8.5 Data augmentation

Not strictly a regularizer in the parameter sense, but functionally similar: augmenting training images with random crops, flips, color jitter, and so on effectively gives the network more data, all with the same labels. This is one of the single most effective overfitting countermeasures in practice. For text, analogous techniques include token masking and back-translation; for audio, time and frequency masking play the same role.

9 Generalization: The Modern Picture

A century of statistical learning theory had a tidy story to tell: increasing model capacity reduces training error monotonically, but test error follows a U-shaped curve — first dropping as we escape underfitting, then rising as we start overfitting. The optimal model sits at the bottom of the U, comfortably below the data-fitting limit. Deep learning broke this story in dramatic ways.

9.1 The classical bias-variance picture

The textbook account decomposes test error like this:

$$\text{Test error} = \underbrace{\text{Bias}^2}_{\text{underfit}} + \underbrace{\text{Variance}}_{\text{overfit}} + \text{irreducible noise}. \quad (34)$$

A model with too few parameters can't capture the signal (high bias). A model with too many parameters memorizes the noise alongside the signal (high variance). The optimal model balances the two.

9.2 Double descent

Here's an empirically robust observation that contradicts the textbook story. When you keep growing model capacity past the point where the model can fit the training set perfectly (the “interpolation threshold”), test error first *spikes* sharply near that threshold — and then *decreases again* as capacity grows further [17]. Past the threshold, in what's called the “modern interpolating regime,” more parameters don't hurt generalization. They often help it.

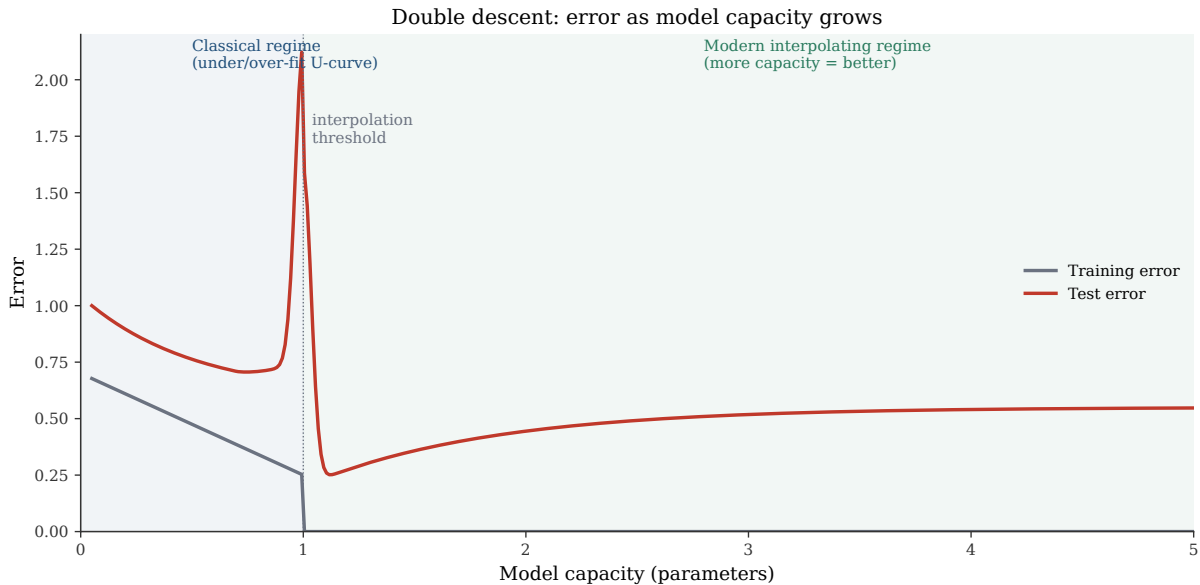


Figure 14: Schematic double-descent curve. In the classical regime (left), test error follows the textbook bias-variance U-shape. At the interpolation threshold, where the model has just enough capacity to fit training data exactly, test error spikes. In the modern interpolating regime (right), more capacity consistently lowers test error — the opposite of what the classical theory predicts.

9.3 Why over-parameterization can help

The classical analysis implicitly assumed the optimizer would find an arbitrary global minimum of the training loss. Modern theoretical work shows that gradient descent on highly over-parameterized networks has *implicit biases* that select *simpler* interpolating functions —

specifically, functions with small weight norms, or equivalently, large margins on the training points. The network has many ways to fit the training data, and SGD prefers the smooth ones.

This observation has reshaped how people think about model design. Instead of carefully sizing the model to match the dataset (the classical recipe), the modern approach is to make the model as large as we can afford and rely on regularization plus SGD’s implicit bias to do the work. The huge language models with billions of parameters trained on trillions of tokens are extreme instantiations of this principle.

9.4 Practical takeaways

- Bigger models tend to be better, given enough data and regularization.
- Overfitting in the traditional sense (training error $\ll$ test error and rising) is still a real phenomenon, but it’s one failure mode among many. The modern regime suggests that the old fear of parameter counts is largely obsolete.
- You still need to watch train/val curves, but “the model has more parameters than data points” is no longer, by itself, evidence of a problem.

10 Implementation: PyTorch and Differentiable Programming

We’ve done the math. Now we ship the code. PyTorch [18] is the most widely used framework for deep-learning research, and it’s built directly on the matrix-calculus principles we developed in the earlier sections.

10.1 The autograd engine

Every tensor in PyTorch can carry a `requires_grad=True` flag. When such a tensor is used in an operation, the framework records the operation in a dynamically-built computation graph. Each recorded node remembers its inputs and knows how to compute the Jacobian-vector product for backpropagation. Calling `.backward()` on a scalar loss traverses this graph in reverse and accumulates gradients into the `.grad` attribute of each leaf tensor.

```
import torch

# Forward pass: graph built automatically.
x = torch.tensor([1.0, 0.1])
W0 = torch.tensor([[1.0, -3.0], [0.2, 1.0]], requires_grad=True)
W1 = torch.tensor([[1.0, -1.0]], requires_grad=True)
y = torch.tensor(0.5)

h = torch.tanh(W0 @ x)
yhat = (W1 @ h).squeeze()
loss = 0.5 * (yhat - y) ** 2

# Backward pass: autograd traces the graph and fills in gradients.
loss.backward()

print(W0.grad)    # exactly the matrix derived by hand in Section 6
print(W1.grad)
```

The gradients populated in `W0.grad` and `W1.grad` will match the manual derivation in Section 6 to the precision of the floating-point representation. That’s reverse-mode autodiff in five lines.

10.2 Modules, optimizers, and the canonical training loop

Realistic PyTorch code uses `nn.Module` objects to encapsulate layers and parameters, and `torch.optim` objects to update them. Here’s the canonical training loop:

```
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, d_in, d_hidden, d_out):
        super().__init__()
        self.fc1 = nn.Linear(d_in, d_hidden)
        self.fc2 = nn.Linear(d_hidden, d_hidden)
        self.fc3 = nn.Linear(d_hidden, d_out)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)

model = MLP(784, 256, 10).cuda()
opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)
crit = nn.CrossEntropyLoss()
```



```

for epoch in range(num_epochs):
    for x, y in train_loader:
        x, y = x.cuda(), y.cuda()
        opt.zero_grad()           # clear last step's gradients
        logits = model(x)         # forward pass; graph built
        loss = crit(logits, y)    # softmax + CE, fused for stability
        loss.backward()           # backward pass; .grad filled in
        opt.step()                # parameter update

```

Three things in this loop are worth pointing out:

- `opt.zero_grad()`: gradients are *accumulated* into `.grad` by default, which allows flexible patterns like gradient accumulation across micro-batches. Forgetting to call `zero_grad` is probably the most common PyTorch bug.
- `crit(logits, y)` takes raw logits, not probabilities. The cross-entropy + softmax combination is computed in a numerically stable fused form, leveraging the elegant Equation (16).
- There is no explicit backward code anywhere. Every operation inside `forward` is differentiable, and autograd takes care of the rest.

10.3 Memory: training vs. inference

Recall that the backward pass needs every intermediate activation cached during the forward pass. For a model with L layers and a batch of B examples, this memory footprint scales as $\mathcal{O}(B \cdot L \cdot d)$, where d is the layer width. For large models, this dominates the GPU memory budget by a huge margin.

But at inference time — when we’re generating predictions, evaluating on a test set, or deploying to production — we don’t need gradients at all. PyTorch provides `torch.no_grad()` as a context manager that disables autograd entirely, freeing the framework from caching intermediates:

```

model.eval()                # disables dropout, switches BN to running stats
with torch.no_grad():
    for x in test_loader:
        logits = model(x.cuda())
        # No graph is built. No activations cached. Memory ~= forward pass
        only.

```

This typically reduces memory consumption by an order of magnitude and gives a modest speedup (the framework can skip all its bookkeeping). For very large models, the memory savings make the difference between a model that fits in GPU memory and one that doesn’t.

Other memory tricks for training.

- **Gradient checkpointing** [19]: recompute, rather than cache, selected intermediate activations during the backward pass. Trades extra FLOPs for less memory. This is standard practice for training models that wouldn’t otherwise fit in memory.
- **Mixed-precision training**: store weights and activations in fp16 or bf16 rather than fp32, with master weights still in fp32. This roughly halves memory consumption at minimal accuracy cost.
- **Activation offloading**: park cached activations on CPU memory or even disk between the forward and backward passes. Useful at extreme scale, where every byte of GPU memory is precious.

10.4 From SGD to LLMs: the same recipe

Every state-of-the-art system out there — ResNets, transformers, vision transformers, foundation models with hundreds of billions of parameters — uses exactly the procedure developed in this report. Larger models, more data, more compute, and a few clever architectural tricks (residual connections, attention, mixture-of-experts), but the math underneath is exactly what we’ve covered: the chain rule applied to a composition of differentiable layers, parameters updated by a variant of Adam, regularized by weight decay and dropout, initialized by He or Xavier, and trained by reverse-mode automatic differentiation. The contemporary AI boom is, mathematically, the patient industrial application of the principles in this document.

11 Closing Thoughts

We started with a 30-year-old problem (perceptrons can't learn XOR) and ended with the engine of modern foundation models. The intervening developments — backpropagation, ReLU, principled initialization, batch normalization, residual connections, adaptive optimizers, attention — are each individually elegant, but the cumulative power they give us comes from the fact that they all slot into the same simple framework: a differentiable function, a loss, a gradient, and an optimizer.

What we haven't covered is at least as much as what we have. Convolutional architectures and the inductive biases they encode for images. Recurrent networks and the attention-based architectures that succeeded them. Graph neural networks. Generative modeling — variational autoencoders, GANs, diffusion. Reinforcement learning's interaction with deep models. The engineering challenge of distributed training across thousands of accelerators. The open scientific question of how and why large neural networks actually generalize. Every one of these areas builds on the foundations we've established here. Once the matrix calculus is internalized, what looks like a research frontier turns out, more often than not, to be a particular and tasteful application of the chain rule.

References

- [1] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.
- [2] M. Minsky and S. Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.
- [3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [4] K. Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.
- [5] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *NeurIPS*, 2012.
- [6] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *AISTATS*, 2010.
- [7] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *ICCV*, 2015.
- [8] B. T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.
- [9] T. Tieleman and G. Hinton. Lecture 6.5 — RMSProp: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning, 2012.
- [10] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [11] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
- [12] I. Loshchilov and F. Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.
- [13] L. N. Smith. Cyclical learning rates for training neural networks. In *WACV*, 2017.
- [14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 15(56):1929–1958, 2014.
- [15] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.
- [16] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry. How does batch normalization help optimization? In *NeurIPS*, 2018.
- [17] M. Belkin, D. Hsu, S. Ma, and S. Mandal. Reconciling modern machine-learning practice and the classical bias-variance trade-off. *PNAS*, 116(32):15849–15854, 2019.
- [18] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.
- [19] T. Chen, B. Xu, C. Zhang, and C. Guestrin. Training deep nets with sublinear memory cost. *arXiv:1604.06174*, 2016.